

第 9 卷 Python 互补卷

考试速查：主查 Python 是否值得用：大整数、字典集合、heapq、deque、itertools、简单模拟和状态搜索；复杂图/区间仍优先 C++。

Contents

第 9 卷 Python 互补卷	2
考试速查定位	2
Python 互补卷使用原则	2
PY-00 Python 互补使用路由	2
Python 特别适合的任务	3
Python 明显不适合的任务	3
PY-01 Python 语法、输入输出与基础容器	4
输入分隔方式总表	5
空格和换行等价的例子	5
行边界有意义的例子	6
混合 token 和整行	6
EOF 读到输入末尾就停止	6
基础写法速查	8
1-index 数组	8
二维数组	8
多关键字排序	8
字典计数	8
集合判重	8
字符串常用操作	8
简单分隔格式	8
格式化输出	8
PY-02 Python 标准库速查	9
常用片段	11
heapq 最小堆和最大堆	11
二分	11
组合枚举	11
数学函数	11
PY-03 Python 暴力、记忆化与 DP 原型	12
记忆化搜索通用骨架	13
例 1: 快速求和 P1874 的 Python 记忆化版	13
例 2: 编辑距离的 Python 稳定版	14
BFS 状态搜索	14
PY-04 Python 常用算法模板	15
BFS 最短路	16
Dijkstra 非负权最短路	17
DSU 并查集	17
树状数组	17
Python int 位集做子集和	18
二分答案	18
PY-05 Python 提交风险清单	19
提交前检查表	20
Python 和 C++ 的临场选择	20
PY-06 Python 核心语法速查	21

控制流	22
函数、全局变量与递归	22
range、enumerate、zip	23
列表推导式与二维数组	23
切片与字符串	23
排序 key 与 lambda	23
bytes 和 str	23
Python 与 C++ 的除法/取模差异	24

第 9 卷 Python 互补卷

考试速查定位

第 9 卷 Python 互补卷

- **这是第几卷：** 09。
- **主要查什么：** 主查 Python 是否值得用：大整数、字典集合、heapq、deque、itertools、简单模拟和状态搜索；复杂图/区间仍优先 C++。
- **什么时候翻：** 只有 Python 明显更省时翻，主力仍建议 C++。
- **题面关键词：** 大整数；字典/集合；heapq；deque；itertools；输入；递归限制；适用/不适用场景。

自动由 PY 模块重建。定位是 C++ 主战之外的补位工具：高精度、状态记忆化、字符串和小数据部分分。

Python 互补卷使用原则

判断	建议
C++ 模板稳、数据大、卡常	用 C++
高精度/复杂状态/字符串解析明显省代码	可用 Python
只想先拿部分分	Python 可快速写暴力/记忆化
需要第三方库才方便	不要用，考试不允许

这卷不是第二主语言资料。它的作用是：当 Python 明显降低实现难度时，帮你快速写出可靠版本；其他时候继续用 C++。

PY-00 Python 互补使用路由

模块编号：PY-00

模块名称：Python 适用场景与 C++ 切换路由

标签：Python、语言选择、部分分、标准库、开卷补充、不能使用第三方库

一句话用途：默认仍用 C++；只有当 Python 能明显省掉高精度、复杂状态、字符串解析、暴力记忆化等实现成本时，才把 Python 当作补位工具。

题面触发词：

- 高精度、大整数、整数位数很长，但运算次数不巨大。
- 状态天然就是字符串、元组、集合、字典键。
- 输入格式杂、字符串处理多、需要快速拆分统计。
- 小数据部分分、暴力枚举、记忆化搜索。
- 组合枚举、排列枚举、计数器、哈希集合。
- 需要快速写出正确版本，而不是压榨常数。

什么时候用：

- 你明显更熟悉 C++，但这道题 Python 的实现优势足够大。
- 题目允许 Python，且时间限制没有明显卡常。
- 你能用 Python 在 10 分钟内写出 C++ 要 25 分钟才稳的版本。
- 大整数直接用 int 能避免手写高精度。
- 状态可以直接用 tuple / str / frozenset 做字典键。
- 目标是先拿小数据或中等数据部分分。
- 代码以清晰正确为主，复杂度本身已经足够。

不要什么时候用：

- n, m 到 $2e5$ 以上，核心循环很多，且算法常数敏感。
- 重型图论、线段树懒标记、最大流、SCC、LCA 多查询等已经有 C++ 稳模板。
- 需要动态有序集合、排名、前驱后继，Python 标准库没有好用的平衡树。
- 题目卡 $O(n \log n)$ 常数或输入巨大，Python 容易 TLE。
- 需要大量二维数组对象，Python 内存开销远高于 C++。
- 考场 Python 版本不明确，而你用了较新的语法。

复杂度：

- Python 不改变算法复杂度，只改变编码成本和常数。
- 粗略经验：Python 纯循环 $1e7$ 级别已经要谨慎， $3e7$ 以上通常危险。
- 内置函数和标准库通常由 C 实现，排序、哈希、math、大整数比纯 Python 循环更可靠。

依赖的标准容器：

- 内置：list、tuple、dict、set、str、int。
- 标准库：sys、collections、heapq、bisect、itertools、functools、math。
- 禁止依赖第三方库：不要使用 numpy、pandas、sortedcontainers、networkx。

输入如何整理：

```
import sys

data = sys.stdin.buffer.read().split()
it = iter(data)
n = int(next(it))
a = [0] + [int(next(it)) for _ in range(n)]
```

接口：

先判断语言：

0. 默认语言是 C++。
1. C++ 模板已经很稳、数据大、卡常 -> 坚持 C++。
2. Python 能直接省掉大量实现，且数据中小 -> 可以用 Python。
3. 不确定能否过 -> Python 先交部分分，C++ 再冲正解。

Python 特别适合的任务

任务	Python 优势	C++ 对照
高精度整数	int 自动任意精度	要手写 BigInteger
状态记忆化	@lru_cache(None) + tuple 参数	要设计数组/hash 编码
字典计数	Counter/defaultdict	map/unordered_map 写法更长
集合判重	set 直接存 tuple/string	C++ 需要 hash 或结构体比较
排列组合	itertools 一行枚举	C++ 要 DFS 或 next_permutation
字符串解析	split/strip/join/replace 快	C++ string 处理更繁
小数据暴力	代码短，容易先拿分	C++ 稳但更长
子集和位集	Python int 位运算很强	C++ 要 bitset 或手写

Python 明显不适合的任务

任务	风险	优先方案
大图 Dijkstra $m \geq 2e5$	堆和邻接表对象常数大	C++ GRAPH-03
懒标记线段树大量操作	递归/对象/函数调用慢	C++ DS-03
最大流、SCC、复杂树链	模板长且常数高	C++ 图论卷
动态有序集合	标准库没有平衡树	C++ set/multiset
巨大二维 DP	Python int/list 对象开销大	C++ 静态数组或滚动数组
深递归 $> 1e5$	栈风险和函数调用慢	C++ 或 Python 迭代

模板代码：

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
```

```

    if not data:
        return
    it = iter(data)
    n = int(next(it))
    a = [0] + [int(next(it)) for _ in range(n)]
    print(sum(a))

if __name__ == "__main__":
    solve()

```

调用示例:

```

# 高精度直接算
a = int("9" * 100)
b = int("8" * 100)
print(a + b)
print(a * b)

# 状态可以直接用 tuple 做 key
seen = set()
seen.add((3, 5, "mask"))

```

常见坑:

- `input()` 循环读大输入容易慢, 优先 `sys.stdin.buffer.read().split()`。
- Python 的数组下标天然 0-index; 本资料建议普通数组仍手动开 `n+1`, 用 `1..n`。
- 字符串天然 0-index, 字符串题不要强行 1-index, 除非你非常确定。
- `heapq` 是最小堆, 没有删除和修改 `key`; 删除要用懒删除。
- `list.insert`、`pop(0)` 是 $O(n)$; 队列用 `collections.deque`。
- `bisect` 只能高效二分, 不能高效中间插入。
- `lru_cache` 会占内存, 状态过多时可能 MLE。
- `sys.setrecursionlimit` 只提高限制, 不保证深递归安全。

暴力/部分替代:

- 先用 Python 写小数据精确解: 排列、子集、DFS、BFS、记忆化。
- 如果 Python 正解可能 TLE, 可以先交 Python 部分分, 再切回 C++ 冲满分。
- 对构造题, Python 可先写合法兜底输出。
- 对计数题, 不取模且答案很大时 Python 先直接输出大整数。

最小测试样例:

输入
3
10 20 30

输出
60

PY-01 Python 语法、输入输出与基础容器

模块编号: PY-01

模块名称: Python 考场语法与容器速查

标签: Python、IO、list、dict、set、str、tuple、排序、格式化输出

一句话用途: 当你决定用 Python 时, 用这一页快速抄输入输出、1-index 数组、字典集合、字符串和格式化输出写法。

题面触发词:

- 输入很大、需要快速读。
- 需要数组、二维数组、字符串、字典计数、集合判重。
- 需要排序、多关键字排序、去重。
- 输出小数、对齐、拼接多行答案。

什么时候用:

- 所有 Python 提交都先从本模块骨架开始。
- 需要把 C++ 思路翻译成 Python 代码。
- 需要快速确认某个容器函数怎么写。

不要什么时候用：

- 不要把 Python 写成复杂工程风格；考场只需要短函数和全局变量。
- 不要为表现语法新特性使用 match、复杂生成器链、装饰器魔法。
- 数据很大时不要逐行 input() 加大量字符串拼接。

复杂度：

- list 末尾 append/pop：均摊 $O(1)$ 。
- dict/set 查询插入删除：均摊 $O(1)$ 。
- sort： $O(n \log n)$ 。
- str 拼接多段：用 ''.join(parts)，不要循环 $s += \text{piece}$ 。
- list.insert(0,x)、pop(0)： $O(n)$ ，队列不要这样写。

依赖的标准容器：

- list：数组、栈、邻接表。
- tuple：多关键字、不可变状态 key。
- dict：映射、稀疏 DP。
- set：去重、访问标记。
- str：字符串处理。
- collections.deque：队列。

输入如何整理：

```
import sys

data = sys.stdin.buffer.read().split()
it = iter(data)

n = int(next(it))
a = [0] + [int(next(it)) for _ in range(n)] # 1-index
```

输入分隔方式总表

先判断输入是 “token 流”，还是 “每行有特殊含义”。

输入形态	Python 推荐	说明
整数/单词由空格、换行、Tab 任意分隔 数据量小、每行格式简单 每一行是一条记录/句子/表达式	sys.stdin.buffer.read().split() input().split() sys.stdin.readline() 或 for line in sys.stdin	全部空白都当分隔符，通常最快 写法短，但大量输入偏慢 保留行边界
字符串可能含空格	line = sys.stdin.readline().rstrip('\n')	不要用 split()
要保留行首/行尾空格 逗号分隔且无引号	只去掉换行：rstrip('\n') line.split(',')	不要用 strip()，它会删空格 真 CSV 有引号时翻 SIM-04 或用 csv 标准 库
JSON/脚本/多行文本	sys.stdin.read()	整段读入，自己解析

口令：

read().split() 会一次性保存所有 token；输入极大或必须流式处理时，改用 buffer.readline()、for line in sys.stdin.buffer`
普通算法题整数输入：read().split()。
行有意义：readline()/for line in sys.stdin。
含空格字符串：不要 split 整行。

空格和换行等价的例子

下面两份输入对 read().split() 完全一样：

```
3
10 20 30
```

```
3 10
20
30
```

代码:

```
import sys

data = sys.stdin.buffer.read().split()
it = iter(data)
n = int(next(it))
a = [0] + [int(next(it)) for _ in range(n)]
```

行边界有意义的例子

```
import sys

n = int(sys.stdin.readline())
lines = [""]
for _ in range(n):
    line = sys.stdin.readline().rstrip("\n") # 保留其他空格
    lines.append(line)
```

混合 token 和整行

读完第一行的 n 后, 再读 n 行句子:

```
import sys

n = int(sys.stdin.readline())
sentences = [""]
for _ in range(n):
    sentences.append(sys.stdin.readline().rstrip("\n"))
```

如果先用 read().split(), 就已经丢掉了所有行边界, 后面不能再恢复整行。

EOF 读到输入末尾就停止

Python 有两种常用 EOF 模式。

Token 流直到 EOF, 最快:

```
import sys

data = sys.stdin.buffer.read().split()
for token in data:
    x = int(token)
    # use x
```

每组两个整数直到 EOF:

```
import sys

data = sys.stdin.buffer.read().split()
out = []
for i in range(0, len(data), 2):
    if i + 1 >= len(data):
        break # 或按题意报错
    n = int(data[i])
    m = int(data[i + 1])
    out.append(str(n + m))
```

```
sys.stdout.write("\n".join(out))
```

每组长度不固定, 例如每组先给 n, 后面跟 n 个数:

```
import sys

data = sys.stdin.buffer.read().split()
```

```

p = 0
out = []
while p < len(data):
    n = int(data[p])
    p += 1
    a = [0]
    for _ in range(n):
        a.append(int(data[p]))
        p += 1
    out.append(str(sum(a)))

sys.stdout.write("\n".join(out))

```

按行读到 EOF:

```

import sys

for line in sys.stdin:
    line = line.rstrip("\n")
    # process line

```

或者显式 readline():

```

import sys

while True:
    line = sys.stdin.readline()
    if line == "": # EOF
        break
    line = line.rstrip("\n")
    # process line

```

注意: 空行是 "\n", 不是 EOF; 如果用 if not line.strip(): break, 会把空行误判成输入结束。

接口:

solve() 里读入、计算、输出。

普通数组开 n+1。

图开 g = [[] for _ in range(n + 1)]。

答案多行先 append 到 out, 最后 '\n'.join(out)。

模板代码:

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    it = iter(data)
    n = int(next(it))

    a = [0] + [int(next(it)) for _ in range(n)]
    a[1:] = sorted(a[1:])

    cnt = {}
    for i in range(1, n + 1):
        cnt[a[i]] = cnt.get(a[i], 0) + 1

    out = []
    out.append(str(sum(a)))
    out.append(" ".join(map(str, a[1:])))
    out.append(str(len(cnt)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    solve()

```

基础写法速查

1-index 数组

```
n = int(input())
a = [0] + list(map(int, input().split()))
for i in range(1, n + 1):
    a[i] += 1
```

二维数组

```
n, m = map(int, input().split())
grid = [[0] * (m + 1)]
for _ in range(n):
    row = [0] + list(map(int, input().split()))
    grid.append(row)
```

多关键字排序

```
items = [(90, 18), (90, 17), (80, 20)]
items.sort(key=lambda x: (-x[0], x[1])) # 第一关键字降序, 第二关键字升序
```

字典计数

```
cnt = {}
for x in a[1:]:
    cnt[x] = cnt.get(x, 0) + 1
```

集合判重

```
seen = set()
state = (x, y, mask)
if state not in seen:
    seen.add(state)
```

字符串常用操作

```
s = " abc,def,ghi "
s = s.strip()
parts = s.split(",")
t = "-".join(parts)
print(s[0], s[-1], s[1:4])
```

简单分隔格式

空白分隔:

```
nums = list(map(int, line.split()))
```

逗号分隔:

```
cells = line.rstrip("\n").split(",")
```

key=value:

```
key, value = line.rstrip("\n").split("=", 1)
```

key: value:

```
key, value = line.rstrip("\n").split(":", 1)
```

```
value = value.lstrip() # 只去掉冒号后的前导空格
```

格式化输出

```
x = 3.1415926
print(f"{x:.2f}") # 3.14
print(f"{123:>6}") # 右对齐宽度 6
print(f"{123:06d}") # 000123
```

调用示例:

```
from collections import deque
```

```
q = deque()
q.append(1)
q.append(2)
print(q.popleft())
```

常见坑:

- `range(1, n)` 不包含 `n`; 1-index 循环写 `range(1, n + 1)`。
- `a = [[0] * m] * n` 是错的, 多行会共用同一个列表。
- `sort()` 原地排序并返回 `None`; `b = sorted(a)` 才是新数组。
- `dict` 不等于有序平衡树; 不能高效求前驱后继。
- `int(next(it))` 中 `next(it)` 是 `bytes`, `int` 可以直接处理, 不必 `.decode()`。
- `read().split()` 会丢掉换行和空行; 行结构重要时不要用。
- `strip()` 会删除行首和行尾空格; 只想删换行用 `rstrip("\n")`。
- `print` 很多次会慢, 答案多行用列表收集后一次输出。

暴力/部分替代:

- 不会优化时, 用 `dict/set` 直接记状态, 先过小数据。
- 字符串题先用 `split/replace/count/find` 写朴素解。
- 排序后扫描能拿一部分时, 不要先写复杂数据结构。

最小测试样例:

输入

```
5
3 1 2 2 5
```

输出

```
13
1 2 2 3 5
4
```

PY-02 Python 标准库速查

模块编号: PY-02

模块名称: 无第三方库条件下的 Python 标准库算法工具

标签: Python、标准库、`heapq`、`bisect`、`itertools`、`collections`、`functools`、`math`

一句话用途: 只用 Python 标准库, 把常见算法工具补齐: 队列、堆、二分、枚举、记忆化、计数、数学函数。

题面触发词:

- Top-K、最小堆、Dijkstra。
- 排序数组二分、插入位置、第一个大于等于。
- 组合、排列、笛卡尔积。
- 计数、默认字典、双端队列。
- 递归状态记忆化。
- `gcd`、组合数、整根、取模快速幂。

什么时候用:

- Python 代码需要堆、队列、二分、组合枚举。
- 想快速写暴力枚举或小数据精确解。
- 需要标准数学函数, 且不能用第三方库。

不要什么时候用:

- 不要用 `bisect + list.insert` 模拟大规模平衡树, 中间插入是 $O(n)$ 。
- 不要用 `itertools.permutations` 处理 $n > 10$ 的全排列。
- 不要滥用 `Counter` 在巨量循环里创建新对象。
- 不要把 `lru_cache` 用在参数包含 `list/set/dict` 的函数上, 它们不可哈希。

复杂度:

- `heapq.heappush/heappop`: $O(\log n)$ 。
- `bisect_left/right`: $O(\log n)$, 但 `list` 中间插入删除 $O(n)$ 。
- `deque.append/popleft`: $O(1)$ 。
- `itertools.permutations`: 枚举量本身是 $n!$ 。
- `lru_cache`: 每个状态算一次, 哈希 `key` 有常数开销。

依赖的标准容器:

- `collections.deque`
- `collections.Counter`
- `collections.defaultdict`
- `heapq`
- `bisect`
- `itertools`
- `functools.lru_cache`
- `math`

输入如何整理:

```
import sys
from collections import deque, defaultdict, Counter
from heapq import heappush, heappop
from bisect import bisect_left, bisect_right
from itertools import combinations, permutations, product
from functools import lru_cache
from math import gcd, isqrt, comb
```

接口:

队列: `deque`

堆: `heapq`, 默认最小堆

二分: `bisect_left` / `bisect_right`

组合枚举: `itertools`

记忆化: `@lru_cache(None)`

数学: `gcd/isqrt/comb/pow(a,b,mod)`

模板代码:

```
from collections import deque, defaultdict, Counter
from heapq import heappush, heappop
from bisect import bisect_left, bisect_right
from itertools import combinations
from functools import lru_cache
from math import gcd, isqrt
```

```
def demo():
    q = deque([1, 2])
    q.append(3)
    first = q.popleft()

    heap = []
    heappush(heap, (5, "a"))
    heappush(heap, (2, "b"))
    smallest = heappop(heap)

    a = [1, 2, 2, 4]
    left = bisect_left(a, 2)
    right = bisect_right(a, 2)

    cnt = Counter(a)
    pos = defaultdict(list)
    for i, x in enumerate(a, 1):
        pos[x].append(i)

    @lru_cache(None)
    def fib(n):
        if n <= 1:
```

```

        return n
    return fib(n - 1) + fib(n - 2)

pairs = list(combinations([1, 2, 3], 2))
return first, smallest, left, right, cnt[2], pos[2][0], fib(10), gcd(12, 18), isqrt(10), pairs

print(demo())

```

常用片段

heapq 最小堆和最大堆

```
from heapq import heappush, heappop
```

```

min_heap = []
heappush(min_heap, 5)
heappush(min_heap, 2)
print(heappop(min_heap)) # 2

```

```

max_heap = []
heappush(max_heap, -5)
heappush(max_heap, -2)
print(-heappop(max_heap)) # 5

```

二分

```
from bisect import bisect_left, bisect_right
```

```

a = [1, 2, 2, 4, 7]
print(bisect_left(a, 2)) # 第一个 >= 2 的 0-index 位置
print(bisect_right(a, 2)) # 第一个 > 2 的 0-index 位置

```

组合枚举

```
from itertools import combinations, permutations, product
```

```

for comb2 in combinations([1, 2, 3, 4], 2):
    print(comb2)

```

```

for p in permutations([1, 2, 3]):
    print(p)

```

```

for bits in product([0, 1], repeat=3):
    print(bits)

```

数学函数

```
from math import gcd, isqrt, comb
```

```

print(gcd(18, 24))
print(isqrt(10))
print(comb(5, 2))
print(pow(2, 10, 1000)) # 2^10 mod 1000

```

调用示例:

```
from collections import Counter
```

```

s = "abacaba"
cnt = Counter(s)
print(cnt["a"])

```

常见坑:

- heapq 的元素如果第一关键字相同, 会继续比较第二项; 第二项不能比较时要加编号。
- bisect 返回 0-index 位置; 如果你的数组手动 1-index, 不要混淆。
- combinations、permutations 返回迭代器, 数量可能爆炸。

- lru_cache 的参数必须可哈希, list 要转成 tuple。
- defaultdict(list) 访问不存在 key 时会创建空 list, 调试时注意字典大小变化。
- math.comb(n,k) 对很大 n 也能算, 但结果巨大时输出和内存仍可能很大。

暴力/部分分替代:

- 组合枚举可直接拿 $n \leq 20$ 或 k 很小的子任务分。
- Counter 可以快速写频率统计, 先过数据较小的统计题。
- lru_cache 可以把暴力 DFS 直接升级为记忆化搜索。

最小测试样例:

输出

(1, (2, 'b'), 1, 3, 2, 2, 55, 6, 3, [(1, 2), (1, 3), (2, 3)])

PY-03 Python 暴力、记忆化与 DP 原型

模块编号: PY-03

模块名称: Python 快速写暴力搜索、记忆化搜索和 DP

标签: Python、DFS、记忆化、lru_cache、DP、部分分、BFS、状态搜索

一句话用途: 疑似 DP 或搜索题时, 先用 Python 写最直白的 DFS/BFS, 再用 lru_cache 或字典记忆化快速降低复杂度。

题面触发词:

- 每一步有若干选择, 问最优值、可行性、方案数。
- n 小、状态复杂、需要先拿部分分。
- 状态由位置、和、上一个选择、mask、字符串等组成。
- 最短步数、迷宫、状态变化。
- C++ 写 hash 编码麻烦, Python tuple key 很自然。

什么时候用:

- 你能写出暴力递归, 但暂时推不出表格式 DP。
- 递归深度不大, 状态数量可控。
- 状态参数可以直接是整数、字符串、tuple。
- 小数据部分分非常明确。

不要什么时候用:

- 递归深度可能超过 $1e5$, Python 函数调用和栈风险很高。
- 状态数量巨大, lru_cache 会占用大量内存。
- 每个状态转移需要遍历很多对象, Python 常数可能 TLE。
- 已经有 C++ 稳模板能更快满分。

复杂度:

- 纯暴力: 分支数的指数级。
- 记忆化: 约等于 状态数 * 每个状态转移代价。
- BFS: $O(\text{状态数} + \text{转移数})$ 。
- Python 里状态用 tuple 做 key, 会有额外哈希常数。

依赖的标准容器:

- functools.lru_cache
- collections.deque
- dict
- set
- tuple
- list

输入如何整理:

```
import sys
from functools import lru_cache
```

```
data = sys.stdin.buffer.read().split()
it = iter(data)
```

```
s = next(it).decode()
target = int(next(it))
```

接口:

DFS 记忆化:

```
@lru_cache(None)
def dfs(可变状态参数):
    先写 base case
    再枚举选择
    return 答案
```

BFS:

deque + set/dict dist

记忆化搜索通用骨架

模板代码:

```
from functools import lru_cache

INF = 10 ** 18

@lru_cache(None)
def dfs(i, state):
    if i == 0:
        return 0

    ans = INF
    # 枚举选择, 把问题变成更小状态
    # ans = min(ans, dfs(i - 1, new_state) + cost)
    return ans
```

例 1: 快速求和 P1874 的 Python 记忆化版

思路:

- 暴力是从左到右切数字。
- 状态写成 dfs(pos, total): 已经处理到字符串位置 pos, 当前和是 total, 后面最少还要多少个加号。
- 如果 total > target, 直接无效。
- 每次枚举下一段数字 s[pos:nxt]。
- 第一次切出来的数字不需要加号; 之后每多切一段需要 +1。为了简单, dfs 返回从当前状态到结束还需要的段数连接代价, 在转移时用 add = 0 if pos == 0 else 1。

完整代码:

```
import sys
from functools import lru_cache

INF = 10 ** 9

def solve():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    s = data[0].decode()
    target = int(data[1])
    n = len(s)

    @lru_cache(None)
    def dfs(pos, total):
        if total > target:
            return INF
        if pos == n:
            return 0 if total == target else INF
```

```

ans = INF
val = 0
for nxt in range(pos + 1, n + 1):
    val = val * 10 + ord(s[nxt - 1]) - 48
    if total + val > target:
        break
    add = 0 if pos == 0 else 1
    ans = min(ans, add + dfs(nxt, total + val))
return ans

ans = dfs(0, 0)
print(-1 if ans >= INF else ans)

if __name__ == "__main__":
    solve()

```

例 2: 编辑距离的 Python 稳定版

思路:

- 状态 $dfs(i, j)$: 把 $a[i:]$ 变成 $b[j:]$ 的最少操作数。
- 如果某个串空了, 只能全部插入或删除。
- 末字符/首字符相等就跳过, 否则三选一。
- Python 递归记忆化适合小数据推模型; $|a|, |b|$ 到 2000 时, 优先写滚动数组 DP, 避免递归深度和 `lru_cache` 内存问题。

完整代码:

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    if len(data) < 2:
        return
    a = data[0].decode()
    b = data[1].decode()
    n, m = len(a), len(b)

    prev = list(range(m + 1))
    for i in range(1, n + 1):
        cur = [0] * (m + 1)
        cur[0] = i
        ai = a[i - 1]
        for j in range(1, m + 1):
            cost = 0 if ai == b[j - 1] else 1
            cur[j] = min(
                prev[j] + 1,      # delete a[i]
                cur[j - 1] + 1,   # insert b[j]
                prev[j - 1] + cost # replace or keep
            )
        prev = cur

    print(prev[m])

if __name__ == "__main__":
    solve()

```

BFS 状态搜索

```
from collections import deque
```

```

def bfs(start, target):
    q = deque([start])
    dist = {start: 0}
    neighbors = {
        1: [2, 3],

```

```

    2: [4],
    3: [4],
    4: [],
}
while q:
    cur = q.popleft()
    if cur == target:
        return dist[cur]
    for nxt in neighbors.get(cur, []):
        if nxt not in dist:
            dist[nxt] = dist[cur] + 1
            q.append(nxt)
return -1

```

调用示例:

```

# 把 list 状态转 tuple, 才能放进 set/dict/lru_cache
state = [1, 2, 3]
key = tuple(state)
seen = {key}

```

常见坑:

- @lru_cache(None) 放在函数定义上一行; 函数参数必须可哈希。
- base case 要先写, 避免递归无限下去。
- Python 递归深度默认较小, 可写 `sys.setrecursionlimit(1000000)`, 但深递归仍不稳。
- lru_cache 缓存不会自动释放, 状态太多时会 MLE。
- 字符串切片会复制; 高频大切片时尽量传下标。
- BFS 的 set 里不要放 list, 改放 tuple。

暴力/部分分替代:

- 先写无记忆化 DFS, 确认样例正确。
- 把 DFS 函数中真正决定未来的可变参数留下, 加 @lru_cache(None); 如果参数里有 list/set/dict, 先转成 tuple/frozenset。
- 状态太大无法数组化时, 用 Python 字典/缓存先拿中等分。
- 如果 Python 记忆化满分不稳, 保留思路后翻译成 C++ 数组 DP。

最小测试样例:

快速求和输入
99999
45

输出
4

编辑距离输入
sfdqxbw
gfdgw

输出
4

PY-04 Python 常用算法模板

模块编号: PY-04

模块名称: Python 可快速复用的算法与数据结构模板

标签: Python、BFS、Dijkstra、DSU、树状数组、子集和、排序、二分、图

一句话用途: 在 Python 适合的中小规模题里, 直接抄这些短模板; 大规模卡常题仍优先 C++。

题面触发词:

- 无权最短路、网格 BFS。
- 非负权最短路, 但规模不太卡。

- 连通性、合并集合。
- 动态前缀和、逆序对、排名。
- 子集和可行性、大整数位运算。
- 排序后二分、答案二分。

什么时候用：

- 数据规模中等，Python 复杂度足够。
- C++ 模板较长，Python 可以更快写出正确版本。
- 目标是快速拿分或验证思路。

不要什么时候用：

- 数据规模巨大且时间限制严格。
- 需要复杂线段树、最大流、重型图论。
- 需要稳定通过所有强数据，且你已有 C++ 模板。

复杂度：

- BFS: $O(n + m)$ 。
- Dijkstra: $O((n + m) \log m)$ 。
- DSU: 近似 $O(1)$ 。
- 树状数组: 单次 $O(\log n)$ 。
- Python int 位集子集和: 约 $O(n * \text{sum} / \text{word_size})$, 常数由底层大整数优化承担。

依赖的标准容器：

- list
- deque
- heapq
- bisect
- set
- dict

输入如何整理：

```
import sys

data = sys.stdin.buffer.read().split()
it = iter(data)
n = int(next(it))
m = int(next(it))
g = [[] for _ in range(n + 1)]
for _ in range(m):
    u = int(next(it))
    v = int(next(it))
    g[u].append(v)
    g[v].append(u)
```

接口：

```
BFS: dist = bfs(n, g, s)
Dijkstra: dist = dijkstra(n, g, s)
DSU: find(x), union(a,b)
树状数组: add(i,v), sum(i), range_sum(l,r)
```

BFS 最短路

模板代码：

```
from collections import deque

def bfs(n, g, s):
    dist = [-1] * (n + 1)
    dist[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if dist[v] == -1:
```

```

        dist[v] = dist[u] + 1
        q.append(v)
    return dist

```

Dijkstra 非负权最短路

```

from heapq import heappush, heappop

```

```

INF = 10 ** 30

```

```

def dijkstra(n, g, s):
    dist = [INF] * (n + 1)
    dist[s] = 0
    heap = [(0, s)]
    while heap:
        d, u = heappop(heap)
        if d != dist[u]:
            continue
        for v, w in g[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heappush(heap, (nd, v))
    return dist

```

DSU 并查集

```

class DSU:
    def __init__(self, n):
        self.fa = list(range(n + 1))
        self.sz = [1] * (n + 1)

    def find(self, x):
        while x != self.fa[x]:
            self.fa[x] = self.fa[self.fa[x]]
            x = self.fa[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.sz[a] < self.sz[b]:
            a, b = b, a
        self.fa[b] = a
        self.sz[a] += self.sz[b]
        return True

```

树状数组

```

class BIT:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)

    def add(self, i, v):
        while i <= self.n:
            self.bit[i] += v
            i += i & -i

    def sum(self, i):
        res = 0

```

```

while i > 0:
    res += self.bit[i]
    i -= i & -i
return res

def range_sum(self, l, r):
    if l > r:
        return 0
    return self.sum(r) - self.sum(l - 1)

```

Python int 位集做子集和

这个是 Python 的强项之一，适合非负权值、目标和不太大。

```

def subset_sum_possible(a, target):
    bits = 1
    mask = (1 << (target + 1)) - 1
    for x in a:
        if 0 <= x <= target:
            bits |= bits << x
            bits &= mask
    return (bits >> target) & 1

```

二分答案

```

def first_true(lo, hi, check):
    while lo < hi:
        mid = (lo + hi) // 2
        if check(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

```

调用示例：

```

n = 5
dsu = DSU(n)
dsu.union(1, 2)
print(dsu.find(1) == dsu.find(2))

```

常见坑：

- Python 图邻接表对象开销大，m 很大时优先 C++。
- heapq 没有 decrease-key，用“新距离入堆 + 弹出时跳过旧状态”。
- 树状数组仍按 1-index；i = 0 会死循环。
- Python 类方法调用有常数，极限卡常时可以改成函数和列表。
- 子集和位集只适合非负整数；有负数要先偏移或换方法。
- INF 要足够大，Python 不溢出，但别用浮点 inf 混整数。

暴力/部分分替代：

- 图最短路：无权先 BFS；带权小图可先 Floyd 或 DFS 枚举。
- 连通性：不会复杂图算法时，DSU 先拿合并查询分。
- 区间求和：树状数组不会写时，先前缀和处理静态区间。
- 子集和：目标小用位集；n 小用枚举。

最小测试样例：

```

BFS 输入
4 3
1 2
2 3
1 4

```

从 1 出发 dist[3] = 2

PY-05 Python 提交风险清单

模块编号: PY-05

模块名称: Python 局限性、TLE/MLE 风险与提交前检查

标签: Python、TLE、MLE、递归、内存、标准库限制、提交检查

一句话用途: 因为主力仍是 C++, 所以用 Python 前先确认它有明显优势; 用 Python 后按清单避免语言常数、内存和标准库缺口导致丢分。

题面触发词:

- 时间限制短。
- n, m 很大。
- 多组数据。
- 需要递归、堆、二分、字典、二维数组。
- 需要浮点格式或大整数。

什么时候用:

- 你准备临场放弃 C++ 改用 Python 前, 必须先翻这一页。
- Python 代码写完后提交前检查。
- 在 C++ 和 Python 之间犹豫时, 用本模块判断风险。
- Python 部分版本准备提交时, 确保不会因为 IO/递归/内存直接 0 分。

不要什么时候用:

- 不要因为 Python 写法短, 就忽略复杂度。
- 不要把“样例过了”当成 Python 不会 TLE 的证据。
- 不要使用任何第三方库。

复杂度:

- 语言不会拯救错误复杂度; Python 对常数更敏感。
- 大循环估算要更保守: $1e7$ 级别谨慎, $3e7$ 以上危险。
- dict/set/tuple 很方便, 但每个状态内存远大于 C++ 数组。

依赖的标准容器:

- `sys.stdin.buffer`
- `sys.stdout.write`
- `list`
- `dict`
- `set`
- `tuple`
- `deque`
- `heapq`

输入如何整理:

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    # 统一在这里解析, 不要边读边慢速 input

if __name__ == "__main__":
    solve()
```

接口:

Python 提交前检查:

1. 输入是否用 `buffer`。
2. 输出是否批量。
3. 是否用了第三方库。
4. 最大循环次数是否可接受。
5. 递归深度是否安全。
6. 容器是否会爆内存。
7. `list/deque/bisect/heapq` 是否用对复杂度。

模板代码:

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    out = []
    for x in data:
        out.append(str(int(x)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    solve()
```

提交前检查表

检查项	风险	修法
大输入用 input()	慢	sys.stdin.buffer.read().split(); 超大输入改 buffer.readline() 流式读
循环里频繁 print	慢	out.append 后一次输出
队列用 pop(0)	O(n)	deque.popleft()
中间插入 list	O(n)	换 C++ 平衡树或重新建模
lru_cache 状态过多	MLE	压状态、剪枝、改 C++
深递归	RE/TLE	迭代写法或 C++
二维数组太大	MLE	滚动数组、稀疏 dict、C++
heapq 删除元素	没有直接删除	懒删除或重复入堆
使用 numpy	违规	只用标准库
浮点比较	精度误差	用整数化或 eps

Python 和 C++ 的临场选择

数据特征	建议
n <= 20 状态/枚举	Python 可优先
n <= 40 折半枚举	Python 可写, 但注意枚举量
n <= 2000 字符串 DP	Python 可能可过, 最好滚动数组
n, m <= 1e5 图论	C++ 更稳
高精度输出	Python 优先
动态区间修改查询	C++ 优先
复杂哈希状态搜索	Python 优先拿部分分
强卡常最短路/线段树	C++ 优先

调用示例:

```
# 多行输出不要这样:
# for x in ans:
#     print(x)

# 建议这样:
ans = [1, 2, 3]
sys.stdout.write("\n".join(map(str, ans)))
```

常见坑:

- // 是向下取整, 不是向 0 取整。负数除法要特别小心。
- Python % 的结果符号跟除数方向有关, 和 C++ 负数取模可能不同。
- True/False 可以当 1/0, 但输出格式要按题目要求。
- float 是双精度, 不是高精度实数。
- sys.setrecursionlimit(1000000) 不等于递归一定安全。
- copy() 只是浅拷贝, 二维数组复制要小心。

暴力/部分分替代：

- Python 适合先写清楚的部分分版本，尤其是 DFS/BFS/记忆化。
- 如果估算会 TLE，不要硬冲，用 C++ 正解模板接力。
- 大数据不会做时，Python 也可以写合法兜底输出，但要保证不 RE。

最小测试样例：

输入

1 2 3

输出

1

2

3

PY-06 Python 核心语法速查

模块编号：PY-06

模块名称：Python 控制流、函数、切片、推导式与语法坑

标签：Python、语法、控制流、函数、切片、推导式、lambda、bytes、str、作用域

一句话用途：当你临场决定用 Python 时，用这一页快速确认基础语法怎么写，避免因为缩进、范围、除法、切片和作用域丢分。

题面触发词：

- 需要把 C++ 思路快速翻译成 Python。
- 需要写循环、判断、函数、递归、排序 key。
- 需要处理字符串、字节输入、切片、下标。
- 需要列表推导式或生成二维数组。
- 需要注意 Python 与 C++ 的整数除法/取模差异。

什么时候用：

- 已经根据 PY-00 判断 Python 有明显优势。
- 写 Python 代码前先翻一遍基础语法。
- 把 C++ 模板改成 Python 部分分版本时。
- 调试 Python WA/RE/TLE 时检查语言坑。

不要什么时候用：

- 不要因为语法短就用 Python 硬冲大数据卡常题。
- 不要在考场使用复杂新语法，例如 match、海象运算符、复杂装饰器。
- 不要把这一页当 Python 教材；它是考场速查。

复杂度：

- 语法本身不改变复杂度。
- 切片会复制， $a[l:r]$ 是 $O(r-l)$ 。
- 列表推导式通常比手写 append 略简洁，但仍然按元素循环。
- in list 是 $O(n)$ ， in set/dict 均摊 $O(1)$ 。

依赖的标准容器：

- list
- tuple
- dict
- set
- str
- bytes
- range

输入如何整理：

```
import sys
```

```
data = sys.stdin.buffer.read().split()
it = iter(data)
```

```
n = int(next(it))
s = next(it).decode() # 需要字符串函数时再 decode
```

接口:

控制流: if / elif / else, for, while, break, continue

函数: def solve(): return

排序 key: a.sort(key=lambda x: x[1])

遍历: range, enumerate, zip

推导式: [expr for x in xs if cond]

切片: s[l:r] 左闭右开

模板代码:

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    if not data:
        return

    n = int(data[0])
    a = [0] + [int(x) for x in data[1:1 + n]]

    total = 0
    best = -10 ** 30
    for i in range(1, n + 1):
        x = a[i]
        if x > 0:
            total += x
        elif x == 0:
            continue
        else:
            total -= -x
        best = max(best, total)

    sys.stdout.write(str(best))

if __name__ == "__main__":
    solve()
```

控制流

```
if x < 0:
    print("negative")
elif x == 0:
    print("zero")
else:
    print("positive")

for i in range(1, n + 1):
    if a[i] == 0:
        continue
    if a[i] < 0:
        break

while n > 0:
    n //= 2
```

函数、全局变量与递归

```
import sys
sys.setrecursionlimit(1000000)

ans = 0
```

```
def dfs(u, parent):
    global ans
    ans += 1
    for v in g[u]:
        if v != parent:
            dfs(v, u)
```

注意:

- 函数里只读全局变量不需要 `global`。
- 函数里要给全局变量重新赋值，必须写 `global ans`。
- 深递归仍然可能不稳；树很深时 C++ 更可靠。

range、enumerate、zip

```
for i in range(1, n + 1):
    print(i)

for idx, x in enumerate(a[1:], 1):
    print(idx, x)

for x, y in zip(a, b):
    print(x, y)
```

列表推导式与二维数组

```
a = [int(x) for x in data]
even = [x for x in a if x % 2 == 0]

grid = [[0] * (m + 1) for _ in range(n + 1)]
```

不要写:

```
grid = [[0] * (m + 1)] * (n + 1) # 错, 多行共用同一行
```

切片与字符串

```
s = "abcdef"
print(s[0])      # a
print(s[-1])     # f
print(s[1:4])    # bcd
print(s[:3])     # abc
print(s[3:])     # def
print(s[::-1])   # fedcba
```

切片左闭右开，会复制新对象；大循环里频繁切长字符串可能 TLE。

排序 key 与 lambda

```
items = [(90, 18), (90, 17), (80, 20)]
items.sort(key=lambda x: (-x[0], x[1]))

words = ["bbb", "a", "cc"]
words.sort(key=lambda s: (len(s), s))
```

bytes 和 str

```
data = sys.stdin.buffer.read().split()
x = int(data[0])          # bytes 可以直接转 int
s = data[1].decode()      # 需要字符串方法时转 str

if s.startswith("abc"):
    print(s.upper())
```

Python 与 C++ 的除法/取模差异

```
print(-7 // 3) # -3, 向下取整
print(-7 % 3)  # 2
```

C++ 中 $-7 / 3 == -2$, $-7 \% 3 == -1$ 。如果题目要求 C++ 风格向 0 取整, 可写:

```
def div_trunc(a, b):
    q = abs(a) // abs(b)
    if (a < 0) ^ (b < 0):
        q = -q
    return q

def mod_trunc(a, b):
    return a - div_trunc(a, b) * b
```

调用示例:

```
print(div_trunc(-7, 3))
print(mod_trunc(-7, 3))
```

常见坑:

- Python 靠缩进表示代码块, 不能像 C++ 一样靠 {}。
- `range(1, r)` 不包含 `r`。
- `and`、`or`、`not` 不是 `&&`、`||`、`!`。
- `//` 对负数不是向 0 取整。
- `a = b` 不是复制列表, 只是引用同一个对象; 复制一维列表用 `a = b[:]`。
- 空列表、空字符串、0 都是假; 非空容器一般是真。
- `is` 判断对象身份, 比较数值/字符串通常用 `==`。

暴力/部分分替代:

- 用 Python 写部分分时, 宁可语法朴素, 也不要写复杂推导式。
- 能用 `for i in range(...)` 写清楚, 就不要强行一行流。
- 排序和哈希统计用内置能力, 别手写低效循环。

最小测试样例:

输入

```
5
3 -1 0 4 -2
```

输出

```
6
```